

Module 14: DELETE Statement

The **DELETE** statement is used to remove existing records from a table. It is one of the core DML (Data Manipulation Language) commands and must be used carefully because deleted data may be difficult or impossible to recover.



Introduction

In real-world databases, data often becomes unnecessary.

Examples:

- A customer closes their account
- An employee leaves the company
- A test record needs removal
- Duplicate records must be cleaned

Instead of modifying data, we remove it using:

DELETE



Learning Objectives

After completing this module, you will be able to:

- ✓ Delete single records
- ✓ Delete multiple records
- ✓ Use DELETE with WHERE
- ✓ Understand DELETE vs TRUNCATE
- ✓ Understand DELETE vs DROP
- ✓ Use transactions for safe deletion
- ✓ Avoid common deletion mistakes



Table of Contents

1. What is DELETE?

2. Why DELETE is Important
3. DELETE Syntax
4. Deleting a Single Row
5. Deleting Multiple Rows
6. DELETE with WHERE
7. DELETE Without WHERE
8. DELETE and Foreign Keys
9. DELETE with Transactions
10. DELETE vs TRUNCATE
11. DELETE vs DROP
12. Common Mistakes
13. Best Practices
14. Real-World Examples
15. Summary
16. Practice Questions

1 What is DELETE?

The **DELETE** statement removes rows from a table.

Example

Before:

EmployeeID	EmployeeName
101	Kunal
102	Rahul

Query:

```
DELETE FROM Employees
WHERE EmployeeID = 102;
```

After:

EmployeeID	EmployeeName
101	Kunal

Row removed successfully.

Why DELETE Matters

Used for:

- Data Cleanup
 - Removing Invalid Records
 - Customer Account Deletion
 - Employee Offboarding
 - Test Data Removal
-

2 Why DELETE is Important

Databases continuously grow.

Over time:

```
Old Records  
Duplicate Records  
Invalid Records  
Test Data
```

must be removed.

DELETE provides controlled removal of records.

3 DELETE Syntax

Basic syntax:

```
DELETE FROM TableName  
WHERE Condition;
```

Example

```
DELETE FROM Employees  
WHERE EmployeeID = 101;
```

General Structure

```
DELETE FROM  
WHERE
```



Deleting a Single Row

Example Table

EmployeeID	EmployeeName
101	Kunal
102	Rahul
103	Aman

Delete Employee 102:

```
DELETE FROM Employees  
WHERE EmployeeID = 102;
```

Result:

EmployeeID	EmployeeName
101	Kunal
103	Aman

Why Use Primary Keys?

Best practice:

```
DELETE FROM Employees  
WHERE EmployeeID = 102;
```

Instead of:

```
DELETE FROM Employees  
WHERE EmployeeName = 'Rahul';
```

Names may not be unique.

5 Deleting Multiple Rows

DELETE can remove multiple rows at once.

Example

```
DELETE FROM Employees  
WHERE Department = 'HR';
```

Before

Employee	Department
Kunal	IT
Rahul	HR
Aman	HR

After

Employee	Department
Kunal	IT

All HR employees removed.

6 DELETE with WHERE

WHERE determines which rows are removed.

Example:

```
DELETE FROM Employees
```

```
WHERE Salary < 30000;
```

Deletes:

```
Employees earning below 30000
```

Example:

```
DELETE FROM Orders  
WHERE OrderDate < '2024-01-01';
```

Deletes old orders.

Common Conditions

```
WHERE EmployeeID = 101
```

```
WHERE Salary < 50000
```

```
WHERE Department = 'IT'
```

```
WHERE JoiningDate < '2025-01-01'
```

DELETE Without WHERE

Most dangerous DELETE operation.

Example

```
DELETE FROM Employees;
```

Result:

All rows removed

Before

EmployeeID

101

102

103

After

0 rows

⚠ Table still exists.

Only data is removed.

Important

```
DELETE FROM Employees;
```

≠

```
DROP TABLE Employees;
```

8 DELETE and Foreign Keys

Foreign Keys protect related data.

Example

Departments

DepartmentID
1

Employees

EmployeeID	DepartmentID
101	1

Attempt:

```
DELETE FROM Departments
WHERE DepartmentID = 1;
```

Result:

Foreign Key Constraint Error

Reason:

Employees still reference Department 1.

Solutions

Delete Child Records First

```
DELETE FROM Employees
WHERE DepartmentID = 1;
```

Then:

```
DELETE FROM Departments
WHERE DepartmentID = 1;
```

Use Cascade Delete

Advanced topic.

```
ON DELETE CASCADE
```

Automatically deletes related records.

DELETE with Transactions

Professional databases use transactions.

Example

```
BEGIN TRANSACTION;  
  
DELETE FROM Employees  
WHERE Department = 'IT';
```

Check Results

```
SELECT *  
FROM Employees;
```

Undo

```
ROLLBACK;
```

Or Confirm

```
COMMIT;
```

Why Transactions?

Protect against accidental deletion.

10 DELETE vs TRUNCATE

A very common interview question.

DELETE

Removes rows.

Syntax:

```
DELETE FROM Employees;
```

Characteristics:

- ✓ Can use WHERE
 - ✓ Logs row-by-row
 - ✓ Can rollback
 - ✓ Slower
-

TRUNCATE

Removes all rows.

Syntax:

```
TRUNCATE TABLE Employees;
```

Characteristics:

- ✗ Cannot use WHERE
- ✓ Faster
- ✓ Resets identity values
- ✓ Minimal logging

Comparison

Feature	DELETE	TRUNCATE
Removes Rows	Yes	Yes
WHERE Allowed	Yes	No
Faster	No	Yes
Identity Reset	No	Yes
Row Logging	Yes	Minimal

Example

DELETE:

```
DELETE FROM Employees  
WHERE EmployeeID = 101;
```

TRUNCATE:

```
TRUNCATE TABLE Employees;
```

Removes every row.

DELETE vs DROP

Another common interview question.

DELETE

Removes data only.

```
DELETE FROM Employees;
```

Table remains.

DROP

Removes entire object.

```
DROP TABLE Employees;
```

Deletes:

- Table Structure
- Data
- Indexes
- Constraints

Comparison

Feature	DELETE	DROP
Deletes Data	Yes	Yes
Deletes Table	No	Yes
Table Remains	Yes	No
WHERE Allowed	Yes	No

Example

DELETE:

```
DELETE FROM Employees;
```

Table exists.

DROP:

```
DROP TABLE Employees;
```

Table no longer exists.

1 2 Common Mistakes

Missing WHERE

Bad:

```
DELETE FROM Employees;
```

Result:

```
Entire table emptied.
```

Wrong Condition

Bad:

```
WHERE EmployeeID = 9999
```

No rows deleted.

Ignoring Foreign Keys

Causes constraint errors.

No Backup

Dangerous in production.

No Transaction

Makes recovery difficult.

1 3 Best Practices

Always Run SELECT First

Example:

```
SELECT *  
FROM Employees  
WHERE Department='HR';
```

Verify rows.

Then:

```
DELETE FROM Employees  
WHERE Department='HR';
```

Use Transactions

Example:

```
BEGIN TRANSACTION;
```

Delete.

Review.

```
ROLLBACK;
```

or

```
COMMIT;
```

Use Primary Keys

Good:

```
WHERE EmployeeID = 101
```

Backup Important Data

Before mass deletion.

Avoid DELETE Without WHERE

Unless intentional.

Professional Workflow

Step 1

Verify rows.

```
SELECT *  
FROM Employees  
WHERE Salary < 30000;
```

Step 2

Delete.

```
DELETE FROM Employees  
WHERE Salary < 30000;
```

Step 3

Verify.

```
SELECT *  
FROM Employees;
```

Real-World Examples

Delete Test Records

```
DELETE FROM Customers
WHERE CustomerName = 'Test User';
```

Delete Old Orders

```
DELETE FROM Orders
WHERE OrderDate < '2023-01-01';
```

Remove Inactive Users

```
DELETE FROM Users
WHERE IsActive = 0;
```

Delete Duplicate Records

Advanced example using ROW_NUMBER() (later module).

Clean Staging Table

```
TRUNCATE TABLE StagingSales;
```



Summary

In this module, you learned:

- ✓ DELETE Statement
 - ✓ Single Row Deletion
 - ✓ Multiple Row Deletion
 - ✓ WHERE with DELETE
 - ✓ Transactions
 - ✓ Foreign Key Considerations
 - ✓ DELETE vs TRUNCATE
 - ✓ DELETE vs DROP
 - ✓ Safe Deletion Practices
-



Practice Questions

Theory

1. What is DELETE?
 2. Why is WHERE important in DELETE?
 3. What happens if WHERE is omitted?
 4. Difference between DELETE and TRUNCATE?
 5. Difference between DELETE and DROP?
 6. Can DELETE use WHERE?
 7. Can TRUNCATE use WHERE?
 8. Why use transactions with DELETE?
 9. What happens with foreign key constraints?
 10. What are safe deletion practices?
-

Practical Exercises

Task 1

Delete EmployeeID 101.

Task 2

Delete all employees from:

HR Department

Task 3

Delete employees earning:

Less than 30000

Task 4

Delete records using a date condition.

Task 5

Perform deletion inside a transaction.

Task 6

Compare:

DELETE
TRUNCATE
DROP

using sample tables.

Challenge Project

Create:

Departments
Employees

Apply:

- Foreign Keys
- Transactions
- Safe Deletes
- Department Removal Process

while maintaining referential integrity.



Next Module

→ **Module 15: ALTER TABLE Statement**

Topics Covered:

- ALTER TABLE Basics
- ADD COLUMN
- ALTER COLUMN
- DROP COLUMN
- ADD CONSTRAINT
- DROP CONSTRAINT
- Renaming Objects
- Schema Evolution
- Production Database Changes